

MongoDB + Mongoose + Geospatial.

Proyecto: "EcoTrack - Monitoreo de Huella de Carbono".

¿Qué es MongoDB?

MongoDB es una base de datos "NoSQL" (no relacional) orientada a documentos. A diferencia de SQL (tablas y filas), MongoDB almacena la información en "documentos JSON" (o BSON) organizados en colecciones.

SQL es como una planilla de Excel fija (columnas estrictas). MongoDB es como una carpeta de fichas donde cada ficha puede tener datos diferentes.

En JS:

// Documento en MongoDB (JSON)

```
{
  nombre: "Juan Pérez",
  email: "juan@mail.com",
  actividades: [
    { tipo: "transporte", co2: 2.5 },
    { tipo: "electricidad", co2: 5.0 }
  ]
}
```

MongoDB Atlas (Cloud).

Atlas es el servicio en la nube de MongoDB. Permite:

- Crear clusters gratuitos (512MB - M0)
- Acceso desde cualquier lugar
- Backups automáticos
- Monitoreo visual

Ventaja para el proyecto: No instalamos nada local, todo por internet.

Mongoose - El "puente" entre Node.js y MongoDB.

Mongoose es una librería ODM (Object Document Mapper) que nos permite:

1. Definir Schemas - La "forma" que deben tener nuestros documentos
2. Validar datos - Antes de guardar, verifica tipos y reglas
3. Crear relaciones - Como JOINS pero con `populate()`
4. Ejecutar consultas - De forma más amigable que el driver nativo

// SIN Mongoose (crudo)

```
db.collection('usuarios').find({ edad: { $gt: 18 } })
```

// CON Mongoose (más legible)

```
Usuario.find({ edad: { $gt: 18 } })
```

GeoJSON y Consultas Geoespaciales.

GeoJSON es un estándar para representar datos geográficos en JSON.

Tipos principales:

- `Point`: Un punto (ej: ubicación de un usuario)
- `LineString`: Línea (ej: ruta de un viaje)
- `Polygon`: Polígono (ej: zona de la ciudad)

// Ubicación de un usuario (Plaza de Mayo, Buenos Aires)

```
{  
  type: "Point",  
  coordinates: [-58.3816, -34.6037] // [longitud, latitud]  
}
```

Consultas geoespaciales útiles:

Operador	Qué hace	Ejemplo
`\$near`	Encuentra documentos cerca de un punto	Actividades a menos de 1km

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

`\$geoWithin`	Dentro de un área	Usuarios en un barrio
`\$geoIntersects`	Interseca con un área	Rutas que pasan por una zona

Agregaciones (Aggregation Pipeline).

Las agregaciones son como "tuberías" donde los datos pasan por etapas de transformación:

// Pipeline visual explicado:

// [Colección] → (\$match) → (\$group) → (\$sort) → [Resultado]

// Actividades → Filtro por fecha → Sumar CO2 → Ordenar → Total por usuario

Etapas	Función
`\$match`	Filtra documentos
`\$group`	Agrupar y acumular (sum, avg, count)
`\$sort`	Ordena resultados
`\$project`	Selecciona/renombrar campos
`\$lookup`	"JOIN" entre colecciones

Configuración inicial del proyecto.

Crear carpeta del proyecto

```
mkdir ecotrack-backend
```

```
cd ecotrack-backend
```

Inicializar Node.js

```
npm init -y
```

Instalar dependencias

```
npm install express mongoose dotenv cors bcryptjs jsonwebtoken
```

```
npm install -D nodemon
```

Conexión a MongoDB Atlas.

```
// config/db.js
```

```
const mongoose = require('mongoose');
```

```
const connectDB = async () => {
```

```
try {
  await mongoose.connect(process.env.MONGODB_URI);
  console.log('MongoDB Atlas conectado');
} catch (error) {
  console.error('Error de conexión:', error.message);
  process.exit(1);
}
};
```

```
module.exports = connectDB;
```

Variables de entorno (.env):

```
MONGODB_URI=mongodb+srv://usuario:contraseña@cluster.mongodb.net/ecotrack
PORT=5000
```

Creación de Schemas (Modelo de Datos).

```
// models/Actividad.js
const actividadSchema = new mongoose.Schema({
  usuarioid: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Usuario',
    required: true
  },
  tipo: {
    type: String,
    enum: ['transporte', 'electricidad', 'alimentacion', 'residuos'],
    required: true
  },
  detalles: {
    // Para transporte
    vehiculo: { type: String, enum: ['auto', 'moto', 'bus', 'bici'] },
    distanciaKm: { type: Number, min: 0 },
    combustible: { type: String, enum: ['nafta', 'diesel', 'electrico'] },
    // Para electricidad
```

```
consumoKwh: { type: Number, min: 0 },
// Para alimentacion
tipoComida: { type: String, enum: ['carne', 'vegetariano', 'vegano'] },
// Para residuos
kgResiduos: { type: Number, min: 0 }
},
ubicacion: {
  type: { type: String, enum: ['Point'], default: 'Point' },
  coordinates: {
    type: [Number],
    required: true,
    validate: {
      validator: function(coords) {
        return coords.length === 2 &&
          coords[0] >= -180 && coords[0] <= 180 &&
          coords[1] >= -90 && coords[1] <= 90;
      },
      message: 'Coordenadas inválidas'
    }
  }
},
huellaCO2: { type: Number, default: 0 },
fecha: { type: Date, default: Date.now }
});

// Índice geoespacial (OBLIGATORIO para consultas $near)
actividadSchema.index({ ubicacion: '2dsphere' });

// CALCULO AUTOMATICO antes de guardar
actividadSchema.pre('save', function(next) {
  let co2 = 0;

  switch(this.tipo) {
    case 'transporte':
      if (this.detalles.vehiculo === 'auto') co2 = this.detalles.distanciaKm * 0.12;
      if (this.detalles.vehiculo === 'moto') co2 = this.detalles.distanciaKm * 0.06;
```

```
    if (this.detalles.vehiculo === 'bus') co2 = this.detalles.distanciaKm * 0.03;
    break;
  case 'electricidad':
    co2 = this.detalles.consumoKwh * 0.4;
    break;
  case 'alimentacion':
    if (this.detalles.tipoComida === 'carne') co2 = 7.2;
    if (this.detalles.tipoComida === 'vegetariano') co2 = 2.5;
    if (this.detalles.tipoComida === 'vegano') co2 = 1.5;
    break;
  case 'residuos':
    co2 = this.detalles.kgResiduos * 2.0;
    break;
}

this.huellaCO2 = co2;
next();
});

module.exports = mongoose.model('Actividad', actividadSchema);
```

Ejemplo Práctico.

"FoodTrack - Seguimiento de Alimentos".

```
// models/Comida.js - Diferente estructura para practicar
const comidaSchema = new mongoose.Schema({
  usuarioId: { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario' },
  nombreComida: { type: String, required: true },
  calorías: { type: Number, min: 0 },
  proteínas: { type: Number, min: 0 },
  fecha: { type: Date, default: Date.now },
  ubicacion: {
    type: { type: String, enum: ['Point'] },
    coordinates: [Number]
  }
});
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
});
```

```
comidaSchema.index({ ubicacion: '2dsphere' });
```

```
// Consulta para calorías por día
```

```
const caloríasPorDia = await Comida.aggregate([  
  { $match: { userId: userId } },  
  { $group: {  
    _id: { $dateToString: { format: "%Y-%m-%d", date: "$fecha" } },  
    totalCalorias: { $sum: "$calorias" }  
  } },  
  { $sort: { _id: -1 } }  
]);
```

Requerimientos del Proyecto Ecotrack.

Requerimientos Funcionales.

ID	Requerimiento	Descripción	Peso (%)
E1	Registro de actividades	CRUD completo de actividades (transporte, energía, alimentación)	15%
E2	Cálculo automático de CO2	Sistema que calcula la huella según tipo y detalles	10%
E3	Mapa de calor	Endpoint que devuelve actividades en zona con coordenadas	15%
E4	Rutas óptimas	Sugerir rutas (simulado) con menor huella de carbono	10%
E5	Metas personales	Usuario define meta semanal y se compara con su progreso	10%
E6	Ranking global	Top 10 usuarios con menor huella de carbono	10%

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

E7	Reportes semanales	Endpoint que prepara datos para enviar por email	10%
E8	Documentación API	Swagger o README con ejemplos de uso	10%
E9	Frontend básico	Dashboard con estadísticas y mapa (Leaflet/Mapbox)	10%

Requerimientos no Funcionales.

1. Rendimiento: Las consultas geoespaciales deben ejecutarse en < 500ms
2. Seguridad: Variables de entorno para credenciales
3. Escalabilidad: Uso de índices en campos consultados frecuentemente
4. Compatibilidad: API REST con JSON, frontend responsive

Objetivos de la Clase.

Objetivo General.

Desarrollar un backend completo con MongoDB, Mongoose y consultas geoespaciales, aplicado a un proyecto de impacto ambiental (huella de carbono).

Objetivos Específicos.

1. Tecnológicos:

- Configurar MongoDB Atlas y conectarlo a Node.js
- Diseñar schemas Mongoose con validaciones y hooks (pre-save)
- Implementar consultas geoespaciales (`\$near`, `\$geoWithin`)
- Crear pipelines de agregación para estadísticas

2. Metodológicos:

- Trabajar en pares con Live Share (evaluación en vivo)
- Subir avances incrementales a GitHub
- Documentar decisiones técnicas en informe

3. Transversales:

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- Calcular impacto ambiental de actividades cotidianas
- Promover conciencia ecológica mediante tecnología

Criterios de Evaluación.

Criterio	Porcentaje	Detalle
Trabajo en clase (obligatorio 60%)	60%	Debe completarse durante las 4 horas
Conexión a MongoDB Atlas funcionando	10%	Verificar en logs
3 schemas implementados	15%	Usuario, Actividad, Meta/Reporte
1 consulta geoespacial operativa	15%	Endpoint `/api/actividades/cercanas`
API básica funcionando	10%	Al menos 3 endpoints
Dashboard con mapa simple	10%	Mostrar puntos en mapa
Trabajo en casa	40%	Entrega martes siguiente
Requerimientos E5, E6, E7 implementados	15%	Metas, ranking, reportes
Documentación técnica completa	10%	Informe + README
Despliegue en Vercel/Railway	5%	Enlace funcional
Código limpio y buenas prácticas	10%	Comentarios, estructura, commits
Más puntaje	+5%	Implementar autenticación JWT

Actividad Práctica – Eco Track.

Descripción del Proyecto.

EcoTrack es un Sistema web que permite a los usuarios:

- Registrar actividades diarias y calcular su huella de carbono
- Visualizar en un mapa las zonas con mayores emisiones
- Establecer metas de reducción y competir con otros usuarios

Tecnologías a utilizar:

Capa	Tecnología
Backend	Node.js + Express
Base de Datos	MongoDB Atlas
ODM	Mongoose
Mapas	Leaflet.js (frontend) o Mapbox
Frontend	HTML/CSS/Js/React
Despliegue	Vercel (frontend) + Railway/Render (backend)

Estructura del Proyecto (Mínima).

```
ecotrack/  
├── backend/  
│   ├── config/  
│   │   └── db.js  
│   ├── models/  
│   │   ├── Usuario.js  
│   │   ├── Actividad.js  
│   │   └── Meta.js  
│   ├── routes/  
│   │   ├── actividades.js  
│   │   ├── usuarios.js  
│   │   └── estadisticas.js  
│   ├── controllers/  
│   │   ├── actividadController.js  
│   │   └── usuarioController.js  
│   ├── middleware/  
│   │   └── auth.js (opcional)  
│   └── .env  
└── server.js  
├── frontend/  
│   ├── index.html  
│   ├── css/  
│   │   └── style.css  
│   ├── js/  
│   │   ├── mapa.js  
│   │   └── api.js
```

| └─ assets/
| └─ docs/
| └─ INFORME_TECNICO.md
└─ README.md

Endpoints Mínimos Requeridos (API REST).

Método	Endpoint	Descripción
POST	`/api/actividades`	Crear actividad (calcula CO2 automático)
GET	`/api/actividades/usuario/:id`	Listar actividades de un usuario
GET	`/api/actividades/cercanas?lat=&lng=&distancia=`	GeoNear: actividades en radio
GET	`/api/usuarios/:id/huella`	Total CO2 del usuario (con agregación)
GET	`/api/ranking`	Top 10 usuarios menor huella
PUT	`/api/usuarios/:id/meta`	Actualizar meta semanal
POST	`/api/reportes/semanal/:id`	Generar datos para reporte

Entregables específicos para el Martes.

1. Subir a GitHub:

- Repositorio público con todo el código
- README.md con instrucciones de instalación
- `.gitignore` apropiado

2. Informe Técnico (en docs/INFORME_TECNICO.md):

Informe Técnico - EcoTrack

Integrantes

- Nombre Apellido (Rol: backend)
- Nombre Apellido (Rol: frontend)

Arquitectura de la solución

(Diagrama simple o descripción)

Decisiones técnicas importantes

- ¿Por qué eligieron ese índice geoespacial?
- ¿Cómo calcularon huella de cada actividad?
- ¿Qué dificultades tuvieron?

Endpoints documentados

(Tabla con ejemplos de request/response)

Capturas de pantalla

(Mapa funcionando, postman, consola)

Conclusión

(Qué aprendieron, qué mejorarían)

3. Despliegue en Vercel (o similar):

- Frontend funcionando en línea
- Backend desplegado en Render/Railway

Guía paso a paso.

- Setup inicial | Crear cuenta MongoDB Atlas, crear cluster, obtener URI
- Configuración proyecto | `npm init`, instalar dependencias, crear .env
- Schemas Mongoose | Implementar Usuario, Actividad con pre-save
- API básica | Crear rutas POST y GET, probar con Postman
- Consulta geoespacial | Implementar `/cercanas` con `\$near`, probar
- Frontend simple + mapa | HTML con Leaflet, cargar marcadores
- Completar y subir | Commit a GitHub, compartir avance en ZIP

Cómo compartir el avance parcial (obligatorio antes de finalizar la clase).

1. Comprimir la carpeta del proyecto (excluyendo `node\_modules`)
2. Subir a Google Drive / OneDrive
3. Enviar enlace por la plataforma educativa

4. También enviar enlace al repositorio GitHub

Entrega FINAL: martes anterior a la próxima clase.

- Repositorio completo (backend + frontend)
- Informe técnico en Markdown
- Enlace de despliegue funcionando
- Criterio de aprobación: 70% de requerimientos cumplidos + informe completo.